

Enda Hamilton-Fitzpatrick – 40129192 – ehamiltonfitzpatrick01@qub.ac.uk

GitLab URL - <https://gitlab.eecs.qub.ac.uk/40129192/academics-webdevproject.git>

Development Design Decisions and Software Design Patterns

This web application for academic progression was developed using a modular and scalable Node.js and Express architecture, attempting to follow a pattern centred around the enforced separation of concerns between database access (API), routing logic (controllers) and UI (EJS views and JS). With the system separated into two clearly defined layers: the /api folder containing database-facing REST API routes; and the /webapp folder, which handles EJS views, Express route controllers and client-side JavaScript (JS) behaviour for each page.

I have attempted to implement a modular API design and RESTful structure by containing all backend functionality within the /api folder, structured around routes with logical endpoints (e.g. /grades, /student). Each route contains its own validation to protect the DB, as well as error handling and SQL logic. The API is secured using a checkApiKey middleware to enforce access control, ensuring unauthenticated users can't bypass the frontend to hit the backend endpoints, without being authenticated by successfully logging in. Each API file generally follows the Single Responsibility Principles – with grades handling all grades and progression calculations, focused mainly on the student_module table in the DB with JOIN queries used where required. Whilst student.js handles core CRUD operations with validation embedded throughout and messages.js incorporates both cohort and individual messaging logic.

Frontend routes under /webapp are designed to add an extra layer of protection, each one corresponding to an EJS page view and using session-based authentication. These controllers fetch the API endpoints, again ensuring the user is accessing endpoints securely with the secure API key provided (using the getApiConfig() utility function – adhering to Don't Repeat Yourself (DRY) Principles). Whilst each controller ensures role-based access control through middleware functions like requireAdmin / requireStudent – similarly, messaging is locked to the ID of the user as identified in the creation of the session when they log in, thus ensuring information is protected in a personalised and authenticated way. Helper functions like getUpdatedFields(), comparisonHelpers.js and cleanOptionalFields() were used throughout to adhere to DRY, as did validation functions (e.g. validateGradeFields) placed at the top of JS scripts relating to forms – whilst these were all intentionally incorporating defensive programming by mirroring server-side with client-side validation, protecting the API / DB from bad requests where possible.

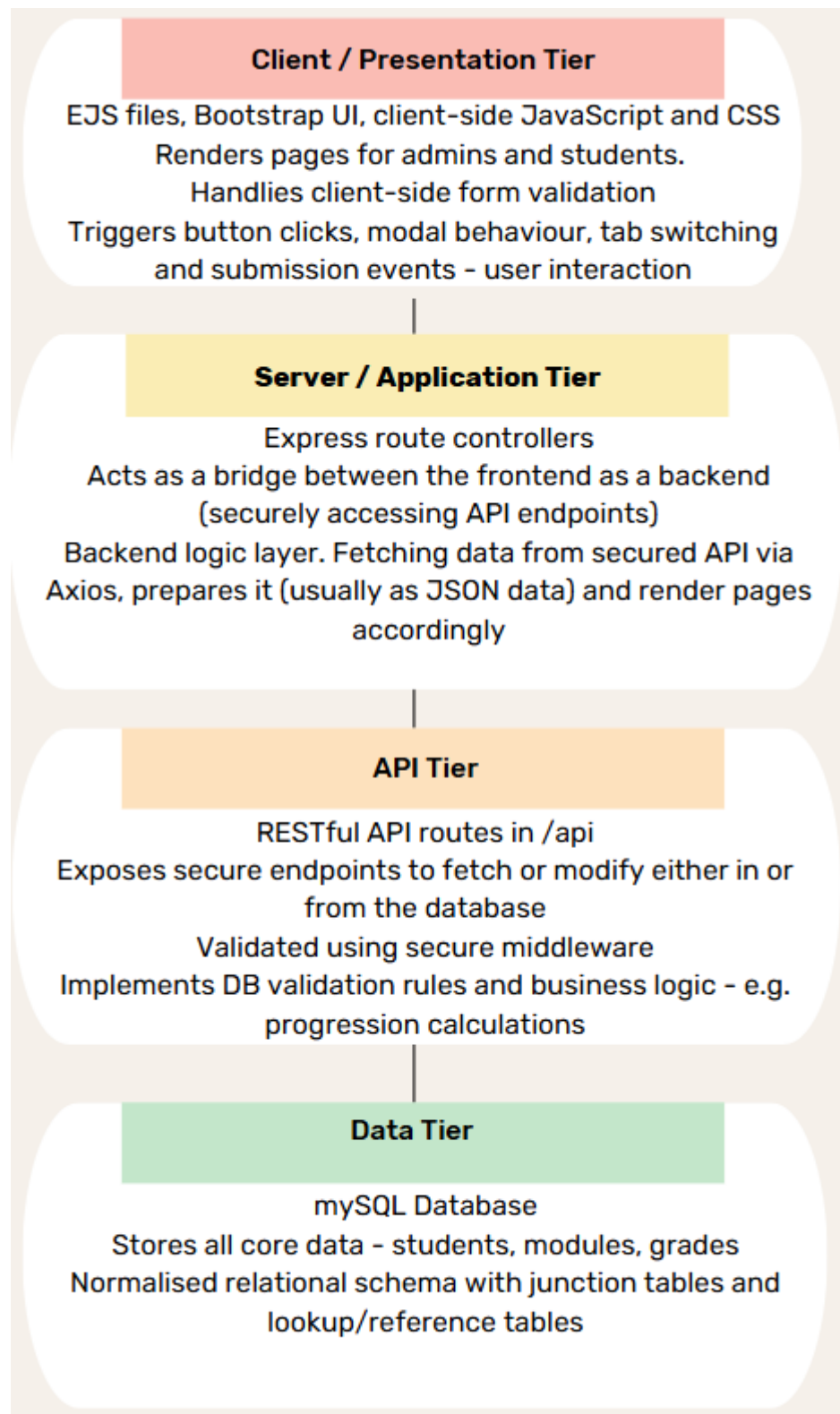
This progressive validation is part of the layered architecture of the application. With data flowing from the mySQL DB to the frontend (and vice-versa) through multiple validation layers – SQL queries and joins serve structured data via Express APIs, with Axios retrieving the data from these endpoints – often using promise requests for efficiency – whilst the client-side javascript applies real-time form validation and returns appropriate responses to the user, with error or success messages clearly displayed in the UI.

Progression calculations are determined within the grades.js backend route using grouped grade data, handling automated validation and rendering these details both to the admin and student-side accordingly. The use of server-side grouping and client-side rendering in this instance enables separation of logic. Similarly, the admin reports/analytics are aggregated without direct DB coupling.

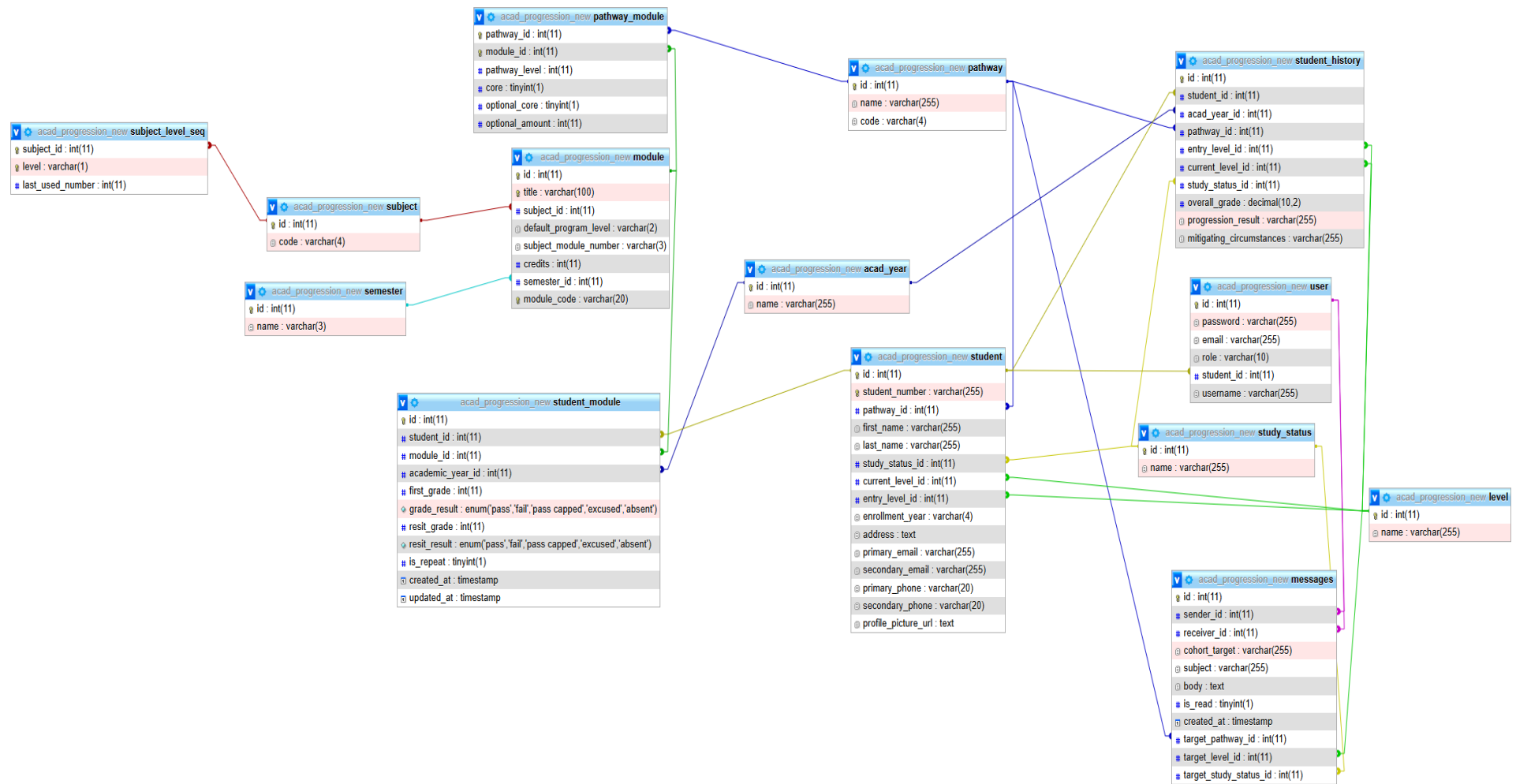
Throughout the system, alerts, badge-based indicators, modals and tabbed interfaces help users interpret feedback clearly. Validation and error messages are specific, status-based and contextual. These UI designs reflect accessibility and UX clarity, particularly for admin-heavy

tasks like student adding/editing, or editing grades/finalising progression. Filterable tables and search bars were incorporated to assist the users with large volumes of records.

System Architecture Diagram



ER Diagram of mySQL Database



List of API Endpoints and REST API Verbs

Route	HTTP Verb	Full endpoint	Description
/acadyear	GET	http://localhost:4000/acadyear	Gets all data from acad_year table
/auth	POST	http://localhost:4000/auth/authenticate	Authenticates user by checking password against email/username
/grades	GET	http://localhost:4000/grades/student/:studentId	Gets all grades for a specific student
/grades	GET	http://localhost:4000/grades/module/:studentId	Gets all grades for a specific module
/grades	GET	http://localhost:4000/grades/	Gets all grades grouped by student
/grades	GET	http://localhost:4000/grades/by-module	Gets all grades grouped by module
/grades	GET	http://localhost:4000/grades/:id	Gets a single grade by its id
/grades	POST	http://localhost:4000/grades	Inserts a student grade into the db
/grades	PUT	http://localhost:4000/grades/:id	Updates a grade by its id
/grades	DELETE	http://localhost:4000/grades/:id	Deletes a student grade by id
/grades	GET	http://localhost:4000/grades/summary/:studentId	Gets a grade summary (credits acquired and avg grade for student
/grades	GET	http://localhost:4000/grades/progrssion/:studentId/:acadYearId	Calculates automatic progression outcome for student by acad year
/grades	POST	http://localhost:4000/grades/finalise-progression	Finalises progression for a student for an acad year
/grades	GET	http://localhost:4000/grades/progression-result/:studentId/:acadYearId	Gets progr result for student for an acad year
/grades	POST	http://localhost:4000/grades/upload-csv	Iterates through an uploaded csv of student grades – parsing each row by student, module and grades. Then doing inserts
/grades	GET	http://localhost:4000/grades/progression/by-pathway-level	Gets all progr data for all students; grouped by acad year, pathway and level
/level	GET	http://localhost:4000/level	Gets all data from level table
/messages	GET	http://localhost:4000/messages/students-with-userid	Gets list of students with user.id included
/messages	POST	http://localhost:4000/messages/individual	Sends a msg to an individual
/messages	POST	http://localhost:4000/messages/cohort	Sends a msg to a targeted cohort
/messages	GET	http://localhost:4000/messages/received-direct/:userId	Gets admins inbox messages
/messages	GET	http://localhost:4000/messages/received/:userId	Gets students received messages
/messages	GET	http://localhost:4000/messages/sent/:userId	Gets sent messages for admin by a user id
/messages	POST	http://localhost:4000/messages/contact-advisor	Sends a msg from student to advisor
/messages	GET	http://localhost:4000/messages/sent-from-student/:userId	Gets a students sent messages
/module	GET	http://localhost:4000/module	Gets all modules including FK related data
/module	GET	http://localhost:4000/module/:id	Gets a module by id

/module	POST	http://localhost:4000/module	Inserts a module into db
/module	PUT	http://localhost:4000/module/:id	Updates a module record by its id
/module	DELETE	http://localhost:4000/module/:id	Deletes a module record by its id
/pathway	GET	http://localhost:4000/pathway	Gets all pathway data
	POST	http://localhost:4000/pathway/assign-module	Assigns a module to a pathway
/semester	GET	http://localhost:4000/semester	Gets all semester data
/student	GET	http://localhost:4000/student	Gets all students
/student		http://localhost:4000/student/details	Gets all students with FK related data
/student	GET	http://localhost:4000/student/:id	Gets a student by its id
/student	GET	http://localhost:4000/student/details/:id	Gets a single student by its id and all FK related data
/student	POST	http://localhost:4000/student	Inserts a student record into db
/student	PUT	http://localhost:4000/student/:id	Updates a student record using id
/student	DELETE	http://localhost:4000/student/:id	Deletes a student record using id
/student	GET	http://localhost:4000/student/by-user/:user_id	Gets a students details by their user id
/student	GET	http://localhost:4000/student/:id/available-modules	Gets available modules for a student based on pathway and level
/student	POST	http://localhost:4000/student/:id/enroll	Inserts into student_module table in db – enrolling student in module
/studystatus	GET	http://localhost:4000/studystatus	Gets all data from study status table
/subject	GET	http://localhost:4000/subject	Gets all data from subject table

References and External Code Sources

- Express.js Framework – Used as framework for building both API and web server
<https://expressjs.com/>
- MySQL2 Node.js – Used for database queries and connection pooling
<https://www.npmjs.com/package/mysql2>
- bcrypt – Used for secure password hashing during user creation -
<https://www.npmjs.com/package/bcrypt>
- Bootstrap - used for styling modals, forms, buttons and layouts in EJS views -
<https://getbootstrap.com/>
- JavaScript Regex – used for client/server validation (e.g. phone numbers) -
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_Expressions
- Session management in express -
<https://expressjs.com/en/resources/middleware/session.html>
- Stack Overflow – used for debugging and troubleshooting - <https://stackoverflow.com/>
- ChatGPT – used for debugging and troubleshooting
- PapaParse – used for parsing CSV file client-side -
<https://www.npmjs.com/package/papaparse>
- Font awesome – used for icons - <https://fontawesome.com/icons>
- Feather – used for icons - <https://feathericons.com/>